

SPM: Concurrency in C++

Academic year 2025-2026

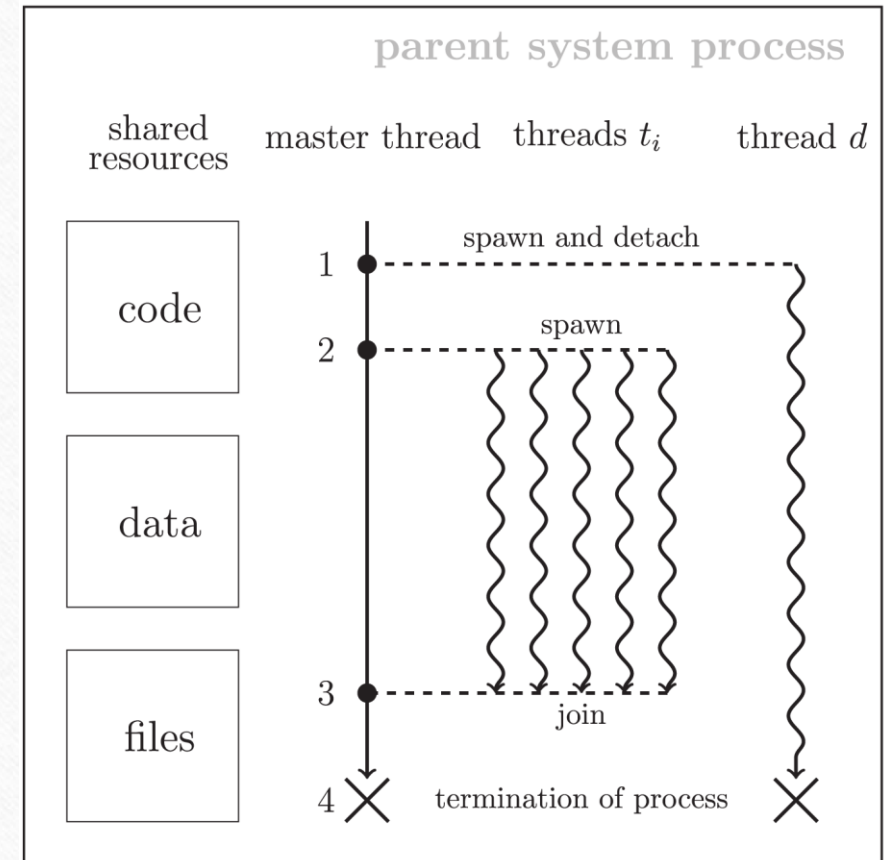
Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Spawning C++ threads
- Mutexes and Condition Variables
- Classic Concurrency Problems
- Future, Promise and Async
- Task objects

Spawning and joining C++ threads

- The master thread can spawn threads, and each thread can spawn threads as well
- The number of spawned threads should roughly match the number of cores (i.e., pay attention to **oversubscription**)
- Threads share process resources
 - each thread has a separate stack
- A thread can be **joined** or **detached** once
- A detached thread cannot be joined
- Joined or detached threads cannot be reused
- All threads must be joined or detached within the scope of their declaration (i.e., before the `std::thread` object will be destroyed)



Spawning and joining C++ threads

- We store thread handles explicitly so that we can later join all spawned threads
- In the code snippet, we use a vector container and the method *emplace_back*, it forwards the constructor arguments and builds the `std::thread` directly in the vector
- Alternatively, we could create a temporary `std::thread` and *move* it into the vector with *push_back*

```
threads.push_back(std::thread(say_hello, id));
```

- The type `std::thread` is *move-only* (i.e., not copyable), i.e., copy constructor and copy assignment operator are deleted (i.e., method-definition = *delete*)
- How to compile:

```
g++ -O3 -std=c++20 spawn-and-join.cpp -o spawn -pthread
```

```
spawn-and-join.cpp > ...
1  #include <stdint> // uint64_t
2  #include <iostream> // std::cout std::endl
3  #include <vector> // std::vector
4  #include <thread> // std::thread
5
6  // this function will be called by the threads (should be void)
7  void say_hello(uint64_t id) {
8      std::cout << "Hello from thread: " << id << std::endl;
9  }
10
11 // this runs in the master thread
12 int main(int argc, char * argv[]) {
13
14     const uint64_t num_threads{(argc==2)?std::stoul(argv[1]):10};
15     std::vector<std::thread> threads;
16
17     // for all threads
18     for (uint64_t id = 0; id < num_threads; id++)
19         // emplace the thread object in vector threads
20         // using argument forwarding, this avoids unnecessary
21         // move operations to the vector after thread creation
22         threads.emplace_back(
23             // call say_hello with argument id
24             say_hello, id
25         );
26
27     // join each thread at the end
28     for (auto& thread: threads)
29         thread.join();
30 }
```


Passing arguments to threads

- Let's consider a simple scalar function that iteratively computes the n-th Fibonacci's number

$$a_n = a_{n-1} + a_{n-2} \quad \text{with initial conditions} \quad a_0 = 0, a_1 = 1$$

- In the code snippet we show two different ways to pass arguments to the thread functions; the second argument is **passed by reference** to store the result using `std::ref`
- Potential pitfalls:
 - The memory passed via a reference (or pointer) has to be persistent during the execution of threads
 - The memory used for variables or objects defined within the loop body where threads are spawned is destroyed when leaving the scope (the destructor is called)

```
fibonacci.cpp > ...
3  #include <vector>    // std::vector
4  #include <thread>    // std::thread
5
6  template <typename value_t>
7  void fibo(value_t n,
8           value_t & result) { // <- here we pass the reference
9      value_t a_0 = 0, a_1 = 1;
10     for (uint64_t index = 0; index < n; index++) {
11         const value_t tmp = a_0; a_0 = a_1; a_1 += tmp;
12     }
13     result = a_0;
14 }
15
16 // this runs in the master thread
17 int main(int argc, char * argv[]) {
18     const uint64_t num_threads = 32;
19     std::vector<std::thread> threads;
20
21     // allocate num_threads many result values
22     std::vector<uint64_t> results(num_threads, 0);
23
24     for (uint64_t id = 0; id < num_threads; id++)
25         threads.emplace_back(
26             // specify template parameters and arguments
27             // fibo<uint64_t>, id, std::ref(results[id])
28
29             // using lambda and automatic type deduction
30             // pay attention to id, must be passed by value!
31             [id, &results]() { fibo(id, results[id]); }
32         );
33     // join the threads
34     for (auto& thread: threads)
35         thread.join();
36     // print the result
37     for (const auto& result: results)
38         std::cout << result << std::endl;
39 }
```


Mutexes

- A mutex is a synchronization mechanism that guarantees **mutual exclusion** when executing a critical section. Its usage restricts the execution of a critical section to a single thread at a time
- A thread locking a mutex prevents other threads from acquiring the mutex. The other threads **wait** for its release (**passive waiting**)

```
#include <mutex>
std::mutex mutex;

// to be called by threads
void some_function(...) {

    mutex.lock()
    // this region is only processed by one thread at a time
    mutex.unlock();

    // this region is processed in parallel
}
```

```
#include <mutex>
std::mutex mutex;

// to be called by threads
void some_function(...) {

    {
        // here we acquire the lock
        std::lock_guard<std::mutex> lock_guard(mutex);
        // this region is locked by the mutex
    } // <- here we release the lock

    // this region is processed in parallel
}
```

- Prefer RAII wrappers (lock_guard / unique_lock) over manual lock()/unlock().

lock_guard vs. unique_lock vs. scoped_lock

- *std::lock_guard* is a RAII wrapper around *std::mutex* that acquires ownership of the mutex upon construction and releases it upon destruction
 - Once constructed, it cannot be explicitly released until it goes out of scope
 - It can only manage a single mutex
- *std::unique_lock* is more flexible and supports lock/unlock if needed
 - It can be moved to transfer ownership of the lock
 - Often used with condition variables
- *std::scoped_lock* manages multiple mutexes simultaneously, acquiring/releasing all of them simultaneously when entering/exiting the scope
 - Helps avoid deadlock when you need to acquire multiple locks together (using a lock ordering strategy)
 - Pay attention to the fact that it accepts 0 mutexes; it simply does nothing
 - *scoped_lock* keeps all mutexes locked for the whole scope; if early unlock is needed, use *unique_lock* instead

Condition Variables (CV)

- A CV lets one or more threads **wait (passively)** until a shared condition becomes true
 - Conceptually, a CV is associated with an **event or condition**. When a thread has determined that the condition is satisfied, it can **notify** one or more waiting threads on the CV to wake them up
 - **Pay attention to spurious wakeups!** The condition must be checked in a loop (typically a while loop) or the **wait** with the predicate version should be used.

Workflow of the signaling thread

1. The signaling thread has to acquire a mutex using either `mutex.lock()` or through a scoped wrapper such as `std::lock_guard` or `std::unique_lock`.
2. While holding the lock, the shared state is modified.
3. The lock is released either explicitly with `mutex.unlock()`, or implicitly by leaving the scope of `std::lock_guard` or `std::unique_lock`.
4. The actual signaling of the condition variable `cv` is performed using `cv.notify_one()`, or `cv.notify_all()`

Note that 3 and 4 can be swapped but is typically slower

Workflow of the waiting thread(s)

1. Acquire a **`std::unique_lock`** using the same mutex as in the signaling phase. Note that `std::lock_guard` cannot be used here.
2. While being locked call either `cv.wait()`, `cv.wait_for()`, or `wait_until()`. The lock is released automatically (**and atomically**).
3. In case (i) `cv` is notified, (ii) the timeout of `cv.wait()` or `cv.wait_for()` expires, or (iii) a spurious wake-up occurs, then the thread is awakened, and the lock is reacquired. At this point, we must recheck the globally shared state (the condition). If the condition is still false, it must continue waiting.

Condition Variables usage pattern

- Typical pattern for the signaling thread

```
{ // notify while holding the lock
  std::lock_guard<std::mutex> lock(mtx);
  ... // modify the predicate (wait_cond)
  cv.notify_one(); // may also use notify_all()
  // correct, but may wake a thread that then blocks again on the mutex
}
```

```
{ // unlock, then notify (better for performance)
  std::lock_guard<std::mutex> lock(mtx);
  ... // modify the predicate (wait_cond)
}
cv.notify_one(); // may also use notify_all()
```

- Typical pattern for the waiter

```
{ // wait in a while loop
  std::unique_lock<std::mutex> lock(mtx);
  ...
  while( !wait_cond ) { // re-check after every wakeup
    cv.wait(lock); //atomically unlock and wait, then re-lock
  }
  ...
}
```

```
{ // wait using a predicate in the wait method (less error-prone)
  std::unique_lock<std::mutex> lock(mtx);
  ...
  cv.wait(lock, [&]() { // return true when it is safe to go on
    return wait_cond; // if false, atomically unlock and wait
  }); // on wakeup re-lock and re-check the wait-cond
  ...
}
```


Producer-Consumer pattern

- Classical concurrency pattern involving two types of threads: `producers`, and `consumers`
- Producers: generate data items and add them to a shared data structure (usually a queue that can have bounded or unbounded capacity)
- Consumers: retrieve data items from the shared data structure (usually one at a time) and consume them doing some work
- Let's see an implementation in which the buffer is unbounded in size (implemented through a `std::deque` container) and the classical implementation with mutex and condition variables.
 - Code: `prod-cons.cpp` (unbounded queue version), `prod-cons-b.cpp` (bounded queue version)

Example: alarm clock

- In the code snippet a thread executes the student function, which checks, in a passive-wait loop, a global variable (*time_for_breakfast*)
- The master thread sets the global variable after a while and notifies the student thread (line 38)
- Remember to use *wait* on a CV either in a while loop or, alternatively, using the *wait* method with a predicate

```
while(wait-condition) {  
    cv.wait(lock);  
}
```

```
cv.wait(lock, [&]() {  
    return !wait-condition;  
});
```

```
alarm_clock.cpp > ...  
4  #include <chrono>           // std::this_thread::sleep_for  
5  #include <condition_variable> // std::condition_variable  
6  // convenient time formats  
7  using namespace std::chrono_literals;  
8  
9  int main() {  
10     std::mutex mutex;  
11     std::condition_variable cv;  
12     bool time_for_breakfast = false; // globally shared state  
13     // to be called by thread  
14     auto student = [&] () -> void {  
15         { // this is the scope of the lock  
16             std::unique_lock<std::mutex> unique_lock(mutex);  
17  
18             // check the globally shared state  
19             while (!time_for_breakfast)  
20                 // lock is released during wait  
21                 cv.wait(unique_lock);  
22  
23             // alternative  
24             //cv.wait(unique_lock,  
25             //         [&]() { return time_for_breakfast; });  
26         } // lock is finally released  
27         std::cout << "Time to make coffee!" << std::endl;  
28     };  
29  
30     // create the waiting thread and wait for 2s  
31     std::thread my_thread(student);  
32     std::this_thread::sleep_for(2s);  
33     { // prepare the alarm clock  
34         std::lock_guard<std::mutex> lock_guard(mutex);  
35         time_for_breakfast = true;  
36     } // here the lock is released  
37     // ring the alarm clock  
38     cv.notify_one();  
39     // wait until breakfast is finished  
40     my_thread.join();  
41 }
```


Attention!

- This is an **incorrect** implementation of the alarm clock example.
- The *sleep_for* calls at lines 20 and 31 have been inserted purposely to make it stall
- What's the problem? Why does it stall?
 - **Race condition**

```
alarm_clock_wrong.cpp > ...
4  #include <chrono>
5  #include <condition_variable>
6  // convenient time formats
7  using namespace std::chrono_literals;
8
9  int main() {
10     std::mutex mutex;
11     std::condition_variable cv;
12     bool time_for_breakfast = false; // globally shared state
13     // to be called by thread
14     auto student = [&] ( ) -> void {
15         { // this is the scope of the lock
16             std::unique_lock<std::mutex> unique_lock(mutex);
17
18             // check the globally shared state
19             while (!time_for_breakfast) {
20                 std::this_thread::sleep_for(1s);
21                 // lock is released during wait
22                 cv.wait(unique_lock);
23             }
24
25             std::cout << "Time to make coffee!" << std::endl;
26         };
27
28         // create the waiting thread and wait for 2s
29         std::thread my_thread(student);
30         std::this_thread::sleep_for(0.5s);
31         time_for_breakfast = true;
32         cv.notify_one();
33         // wait until breakfast is finished
34         my_thread.join();
35     }
36 }
37
```


Multiple-Readers/Single-Writer pattern

- Classical concurrency pattern involving two types of threads: `readers` and `writers`
 - A reader may access the data concurrently with other readers
 - A writer needs exclusive access to the shared data
 - How does this differ from the `Producer-Consumer` pattern?
 - Unlike `Producer-Consumer`, readers and writers have different synchronization constraints
- `std::mutex` serializes all accesses, even read-only ones
- To implement the pattern, the C++17 standard provides `std::shared_mutex`
 - `lock()` / `unique_lock<shared_mutex>`: exclusive access (one writer, no readers)
 - `lock_shared()` / `shared_lock<shared_mutex>`: shared access (multiple readers, no writers)
- `std::shared_mutex` does not guarantee fairness
 - A continuous stream of readers can potentially starve a writer
 - If fairness or writer priority is required, it must be enforced explicitly by the programmer

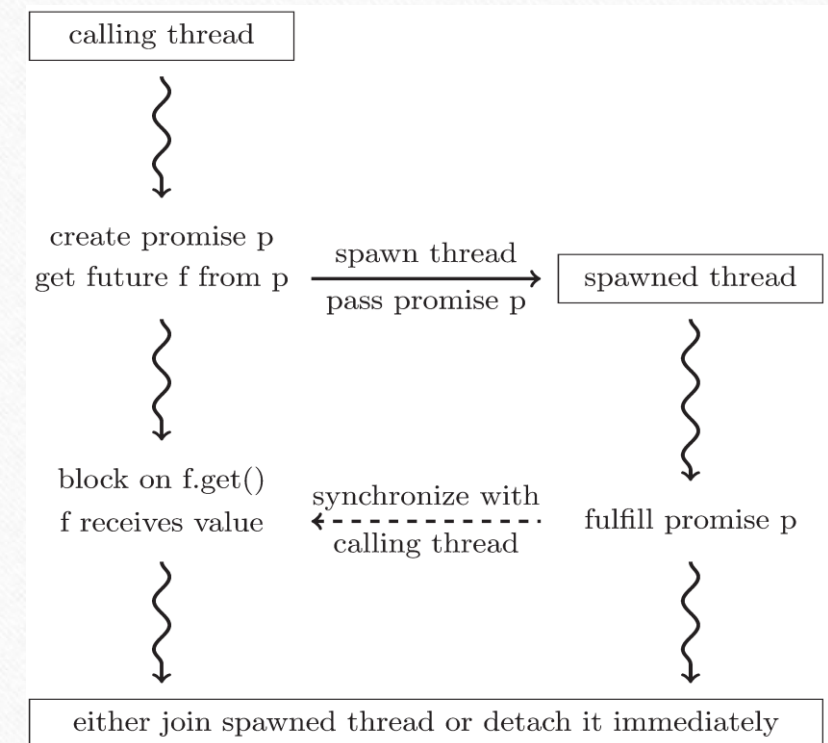
When to use this pattern:

- ✓ many reads, few writes
- ✓ read-side concurrency is valuable
- ✓ shared state is mostly queried, rarely modified

Example: `reader-writer.cpp`

Promises and futures

- C++ provides a mechanism for return-value-passing specifically designed to fit the characteristics of **asynchronous execution**
 - The so-called **promises** that are *fulfilled in the future*
- A promise and a future implement a “**single-assignment channel**” (a one-shot communication mechanism) between two entities, usually two threads
- $s = (p, f)$ where p is a writable view of the state s (the **promise**), f (the **future**), is a readable view of the state s that can be accessed after being signaled (**only once!**) by the promise
- The causal dependency between the *promise* p and the *future* f implements a **synchronization mechanism** between two threads



Promises and futures

- The state $s = (p, f)$ represents the logical channel. It is defined by a promise `std::promise<T> p` for a specific data type T , which has associated the `std::future<T> f = p.get_future()`
 - `std::promise<T> p` represents the write-end of the communication
 - `std::future<T> f` represents the read-end of the communication
- p is passed as *rvalue reference* in the signature of the called function via `std::promise<T> && p;`
 - p has to be moved using `std::move()` from the master thread to the spawned thread since **`std::promise` is not copyable**
- The promise p is fulfilled within the body of the spawned thread by `p.set_value(some_value);`
- We can read the future f in the master thread using `f.get()`, **blocking** its execution until f is signaled by p
- `future::get()` can be called only once on `std::future`; use `std::shared_future` for broadcast/read-many

```
std::promise<int> p;  
auto f = p.get_future();  
std::thread t([p=std::move(p)] mutable {  
    int val ...  
    p.set_value(val); // modifies p  
});  
int x = f.get();  
t.join();
```


One-shot notifications using futures

- CVs are helpful in case many threads perform non-trivial, or repetitive synchronization patterns
- Futures and promises can be used for the so-called *one-shot synchronization* scenario
- Synchronization between the issuing thread and the waiting threads is achieved since the access to a future's values via *f.get()* blocks until the fulfillment of the corresponding promise with *p.set_value()*
- The so-called *shared futures* can be used to **broadcast** a specific value to more than one thread

```
one_shot_alarm_clock.cpp > ...
1 > #include <iostream> // std::cout...
5 // convenient time formats
6 using namespace std::chrono_literals;
7
8 int main() {
9     // create pair (future, promise)
10    std::promise<void> promise;
11    auto shared_future = promise.get_future().share();
12
13    // to be called by thread
14    auto students = [&] (int myid) -> void {
15        // blocks until fulfilling promise
16        shared_future.get();
17        std::cout << "[" << myid <<
18            "]" Time to make coffee!" << std::endl;
19    };
20
21    // create the waiting thread and wait for 2s
22    std::thread my_thread0(students, 0);
23    std::thread my_thread1(students, 1);
24    std::this_thread::sleep_for(2s);
25    promise.set_value();
26
27    // wait until breakfast is finished
28    my_thread0.join();
29    my_thread1.join();
30 }
```


Fibonacci example with promises

- In the code snippet, we show how to pass a single promise to the thread to get the result
 - When using a lambda to call the thread function, pay attention that the promise cannot be passed by reference, it must be moved
 - The lambda must be *mutable* to make the captured entities *non-const* attributes of the closure object, otherwise, it is not possible to call *set_value*

```
Fibopromise.cpp > ...
4  #include <thread>    // std::thread
5  #include <future>    // std::future
6
7  template <typename value_t>
8  void fibo(value_t n,
9           std::promise<value_t> && result) { // <- pass promise
10     value_t a_0 = 0, a_1 = 1;
11     for (uint64_t index = 0; index < n; index++) {
12         const value_t tmp = a_0; a_0 = a_1; a_1 += tmp;
13     }
14     result.set_value(a_0); // fulfill promise
15 }
16
17 // this runs in the master thread
18 int main(int argc, char * argv[]) {
19     const uint64_t num_threads = 32;
20     std::vector<std::thread> threads;
21     // allocate num_threads many result values
22     std::vector<std::future<uint64_t>> results;
23
24     for (uint64_t id = 0; id < num_threads; id++) {
25         // define a promise and store the associated future
26         std::promise<uint64_t> promise;
27         results.emplace_back(promise.get_future());
28         threads.emplace_back(
29             // specify template parameters and arguments
30             // fibo<uint64_t>, id, std::move(promise)
31             // using lambda and automatic type deduction
32             // mutable avoids p (and also id) to be const
33             [id, p{std::move(promise)}]() mutable {fibo(id, std::move(p));}
34         );
35     }
36     // read the futures resulting in synchronization of threads
37     // up to the point where promises are fulfilled
38     for (auto& result: results) std::cout << result.get() << std::endl;
39     // join the threads
40     for (auto& thread: threads) thread.join();
41 }
```

Creating task objects with *std::packaged_task*

- The “*future*” C++ header provides the *std::packaged_task*, a wrapper that turns a callable object into a **task object**
- A task object is a **callable object** with the associated corresponding *future* object handling the return value
 - A callable object is any entity that can be invoked using the function **operator ()**. It includes *std::function* wrapper, functors, lambda expression, *std::bind* expression, regular functions, and function pointers.
- Example:

Assume we want to create a task from this function

```
bool comp(float value, int64_t threshold) {  
    return value < threshold;  
}
```

Create and use the task in this way:

```
// create the task and assign future  
std::packaged_task<bool(float, int64_t)> task(comp);  
auto future = task.get_future();  
  
// call the task with arguments  
task(value, threshold); // WARNING: this is sequential!  
  
// access future object (either true or false)  
std::cout << future.get() << std::endl;
```


Fibonacci example with tasks

- We can create a specific task that calls the fibo function using *std::packaged_task*

```
std::packaged_task<uint64_t(uint64_t)> task(fibo);
```

- Note: *std::packaged_task* is a move-only type
- std::packaged_task* bundles a callable and its future result, but it does not run asynchronously by itself
- We can spawn a thread to execute the task asynchronously by moving the task created with *packaged_task*

```
fibotasks0.cpp x
fibotasks0.cpp > ...
1  #include <iostream> // std::cout
2  #include <cstdint>  // uint64_t
3  #include <vector>   // std::vector
4  #include <thread>   // std::thread
5  #include <future>   // std::future
6
7
8  // traditional signature of fibo
9  uint64_t fibo(uint64_t n) {
10     uint64_t a_0 = 0, a_1 = 1;
11     for (uint64_t index = 0; index < n; index++) {
12         const uint64_t tmp = a_0; a_0 = a_1; a_1 += tmp;
13     }
14     return a_0;
15 }
16
17 // this runs in the master thread
18 int main(int argc, char * argv[]) {
19     const uint64_t num_threads = 32;
20     std::vector<std::thread> threads;
21     // allocate num_threads many result values
22     std::vector<std::future<uint64_t>> results;
23
24     // create tasks, store futures and spawn threads
25     for (uint64_t id = 0; id < num_threads; id++) {
26         std::packaged_task<uint64_t(uint64_t)> task(fibo);
27         results.emplace_back(task.get_future());
28         threads.emplace_back(std::move(task), id);
29     }
30     for (auto& result: results) std::cout << result.get() << std::endl;
31     for (auto& thread: threads) thread.join();
32 }
```

Creating generic task objects

- The problem with the described approach is that the function signature is hard-coded in the template parameter of the `std::packaged_task`
 - Different call signatures produce different task types, making it inconvenient to store and schedule heterogeneous tasks uniformly
- It would be better if all tasks exhibit the same signature, e.g., **void task()**, independently of the callable object to be invoked
- This can be achieved by using a task factory function template (`make_task`), implemented in the `taskFactory.hpp` header file. It returns a pair: a runnable task and its `future`
 - The factory turns arbitrary callables plus arguments into a task object with a uniform signature `void()`
- Purpose of generic task object:
 - Custom scheduling of tasks with a dedicated thread pool
 - Allows to decouple the creation of the task from its execution
 - More control over error handling and task completion

Fibonacci example with generic tasks

- The code is more readable with `make_task`
- A thread executes the task
- The result is embedded in a *`std::future`*

```
Fibotasks.cpp > main(int, char *[])
1  #include <iostream> // std::cout
2  #include <cstdint>   // uint64_t
3  #include <vector>    // std::vector
4  #include <thread>    // std::thread
5  #include <future>    // std::future
6  #include <taskFactory.hpp>
7
8  template <typename value_t>
9  value_t fibo(value_t n) {
10     value_t a_0 = 0, a_1 = 1;
11     for (uint64_t index = 0; index < n; index++) {
12         const value_t tmp = a_0;
13         a_0 = a_1;
14         a_1 += tmp;
15     }
16     return a_0;
17 }
18
19 int main(int argc, char * argv[]) {
20     const uint64_t num_threads = 32;
21
22     std::vector<std::thread> threads;
23     // allocate num_threads many result values
24     std::vector<std::future<uint64_t>> results;
25
26     // create tasks, store futures and spawn threads
27     for (uint64_t id = 0; id < num_threads; id++) {
28         auto [task, future] = make_task(fibo<uint64_t>, id);
29         results.emplace_back(std::move(future)); // std::move mandatory
30         threads.emplace_back(std::move(task));  // std::move prevents a copy
31     }
32     for (auto& result: results) std::cout << result.get() << std::endl;
33     for (auto& thread: threads) thread.join();
34 }
```

The *asynchronous* way

- The “*future*” C++ header provides an out-of-the-box `std::async` function for creating task objects
 - Higher-level interface than explicit thread management for launching tasks asynchronously
- With the default policy, `std::async` may execute the task asynchronously or defer its execution until `get()/wait()`
- However, there are pitfalls to pay attention to:
 - A call to `std::async` **does not necessarily imply that a new thread is spawned**
 - The calling thread might execute the task without spawning a new thread! Default behavior depends on the implementation
 - The execution of the task could be **deferred forever** if the future is not accessed (i.e., `future.get()/wait()`)
 - Creating a temporary future inside a loop may serialize the computation, because the future destructor may wait for task completion
 - To force asynchronous execution, use `std::launch::async`

```
auto future = std::async(fibo, id);
```

```
auto future = std::async(std::launch::async, fibo, id);
```


Fibonacci example with async

- The code snippet of the Fibonacci example is rewritten using the `std::async` construct
- Pay attention to code like the following one:

```
for (uint64_t id = 0; id < num_threads; id++) {  
    auto future = std::async(std::launch::async, fibo, id);  
} // <- here, the destructor of future is called
```

 - The destructor of a temporary future may block until the async task completes, thus serializing the loop
 - Exceptions thrown in the async function are stored and re-thrown on `future.get()`. If you drop the future without calling `get()/wait()`, you never observe them
 - The future must be stored outside the loop body
- The use of `std::async` is more natural when a result or exception propagation is needed. Always keep the returned future (even for void functions) and call `get()` or `wait()`

```
fibosync.cpp > ...  
1  #include <iostream> // std::cout  
2  #include <cstdint>  // uint64_t  
3  #include <vector>   // std::vector  
4  #include <future>   // std::async  
5  
6  uint64_t fibo(uint64_t n) {  
7      uint64_t a_0 = 0, a_1 = 1;  
8      for (uint64_t index = 0; index < n; index++) {  
9          const uint64_t tmp = a_0; a_0 = a_1; a_1 += tmp;  
10     }  
11     return a_0;  
12 }  
13  
14 int main(int argc, char * argv[]) {  
15     const uint64_t num_threads = 32;  
16  
17     std::vector<std::future<uint64_t>> results;  
18     // for each thread  
19     for (uint64_t id = 0; id < num_threads; id++)  
20         // directly emplace the future  
21         results.emplace_back(  
22             std::async(std::launch::async, fibo, id)  
23         );  
24     for (auto& result: results)  
25         std::cout << result.get() << std::endl;  
26 }
```

Mergesort example with async

- Recursively sort [L,M) and [M, R) vector partitions
- Launch an asynchronous task to sort one half with `std::async`, and sort the other half on the current thread, then call `get()` to wait for the completion of the task
- Use a *cutoff* (small range) to control granularity
 - Small ranges are sorted sequentially using `std::sort/stable_sort`
 - The *cutoff* is usually selected empirically
- Avoid **oversubscription** by limiting the depth of recursion
 - `max_depth` is $\lfloor \log_2 \#threads \rfloor$
 - max number of async thread spawned: $2^{\lfloor \log_2 \#threads \rfloor} - 1$
- Merge the halves using either a user-defined temporary buffer of size N (`std::merge` + `std::move`) or `std::inplace_merge` (less allocated memory but usually more overhead)
- The temporary buffer is shared, but each recursive call uses a disjoint subrange, therefore no data race arises on the buffer itself

Example: mergesort-async.cpp

Thread Pool Pattern in C++

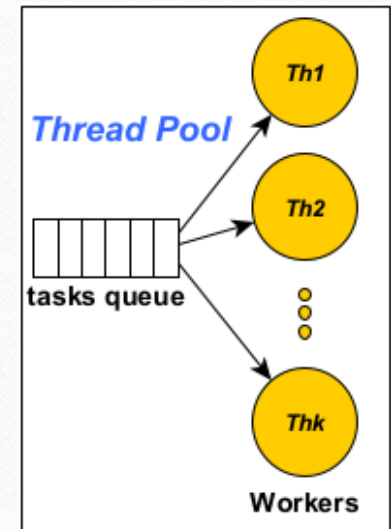
- Instead of creating threads on-demand for each task, we can maintain a pre-spawned set of worker threads that are reused to process all the submitted tasks
- The C++ standard library does not provide a built-in general-purpose thread-pool abstraction
- A common design uses a queue of tasks. Submitted tasks are executed by Worker threads, and results are retrieved through futures

```
threadPool pool(8); // 8 Worker threads in the pool; spawned in the constructor

// function executed by a Worker
auto square = [](uint64_t x) { return x*x; };

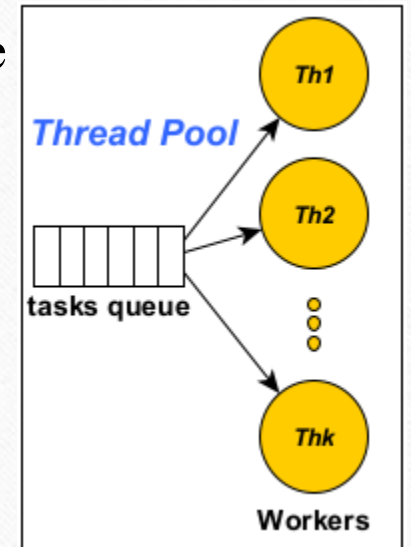
std::vector<std::future<uint64_t>> futures;
for (uint64_t x = 0; x < N; ++x)
    futures.emplace_back( pool.submit(square, x) ); // submit task and store the future

// get() the results outside the loop
```



Thread Pool Pattern

- The Worker threads compete to access the shared task queue
- Each Worker can execute any submitted task (i.e., **Workers are not specialized, but generic executors**)
- The synchronization is managed through *mutex* and *condition variables*, following the Producer-Consumer synchronization pattern
- In the provided implementation (threadPool.hpp) the task queue is **unbounded**
 - What happens if producers generate tasks much faster than Workers can process them?
 - The task queue can grow excessively, leading to high memory usage or even memory exhaustion
- To avoid this issue, we can:
 - Increase the number of Worker threads (if we have enough available cores) to match the producer(s) rate
 - Impose a limit to the task queue size, blocking or throttling producers when the queue is full (**back-pressure**)



Thread Pool in C++

- Usage example of the threadPool implemented in the threadPool.hpp header file
- A task is submitted to the thread pool via `pool.submit(square, x);` (line 20)
- The corresponding future is stored in the *futures* array
- A task is executed by one worker thread. **Parallelism comes from executing different tasks concurrently**

- Need one result consumed by multiple threads? We can use `std::shared_future`
- We can convert a future into a `shared_future` using `share()`
 - `auto sf = pool.submit(task).share();` // `future<T> -> shared_future<T>`
- A `std::shared_future` is copyable, and multiple threads may safely access the same shared result using `get()`

```
square-pool.cpp > main()
1  #include <iostream>
2  #include <threadPool.hpp>
3
4  int main () {
5      // define a thread pool with 8 Workers
6      threadPool pool(8);
7
8      // function to be processed by threads
9      auto square = [](const uint64_t x) {
10         return x*x;
11     };
12
13     // more tasks than threads in the pool
14     const uint64_t ntasks = 128;
15     std::vector<std::future<uint64_t>> futures;
16     futures.reserve(ntasks);
17
18     // enqueue the tasks in a linear fashion
19     for (uint64_t x = 0; x < ntasks; ++x) {
20         futures.emplace_back(pool.submit(square, x));
21     }
22
23     // wait for the results
24     for (auto& future : futures)
25         std::cout << future.get() << std::endl;
26
27     return 0;
28 }
```

Mergesort example with threadPool

- Instead of spawning a new thread, submit a task to the threadPool where threads are reused
- Increased readability and lower overhead
- Cutoff selected empirically (1-4K for int)
 - Essential to avoid creating too many fine-grained tasks, whose scheduling overhead may exceed the useful work
- One subproblem is submitted to the pool, while the other is solved locally by the current thread
 - This keeps the current thread busy, reducing idle time
 - This design also reduces the risk of saturating the pool with tasks that immediately wait for other tasks
 - Parallelism comes from processing independent subarrays concurrently, while each individual sort/merge step remains sequential

Example: mergesort-pool.cpp

Suggested Readings

- Chapter 4 of the «Parallel Programming Concept and Practice» book
- C++ reference <https://en.cppreference.com/w/cpp>
- «C++ High Performance» second edition by B. Andrist, V. Sehr, Packt
 - <https://www.packtpub.com/en-us/product/c-high-performance-9781787120952>
 - Git repo: <https://github.com/PacktPublishing/Cpp-High-Performance-Second-Edition>
 - Chapter 2 of this book provided as additional material
- «C++ Concurrency in Action» second edition by Anthony Williams
 - <https://www.manning.com/books/c-plus-plus-concurrency-in-action-second-edition>